

SkillFocus: Evolving Agent Skills via Capability Decomposition

Ning Wang¹ Zhiren Gong² Bingdong Li^{1*} Peng Yang³ Aimin Zhou^{1,4}

¹East China Normal University; ²Nanyang Technological University;

³Southern University of Science and Technology; ⁴Shanghai Innovation Institute

Abstract

Agent skill evolution seeks to improve reusable procedural guidance for large language model (LLM) agents through iterative revision. Existing methods base each revision mainly on execution trajectories or feedback, leaving recurring behavioral requirements across tasks implicit and tying revision to the behavior of the current skill. We introduce SkillFocus, which decomposes recurring task requirements into a capability space that remains fixed as the skill evolves, separating what tasks require from how the current skill behaves. SkillFocus maps current task outcomes to this space to identify the capability that leaves the most tasks unresolved, then uses that capability to determine what to revise and which evidence to use. Across four benchmarks spanning heterogeneous tasks, SkillFocus achieves the best held-out accuracy on all four, outperforming the strongest competing result by 5.7 points on average while using 24% fewer evolution tokens on average than the closest iterative baseline. Controlled studies further show that capabilities derived from recurring task requirements outperform task-semantic and execution-derived alternatives, while randomizing task-capability assignments reduces final accuracy by up to 20.2 points. Matching evidence to the selected capability increases candidate gain by 4.4 points under prioritized revision.

1 Introduction

Large language model (LLM) agents (Yao et al., 2023) can benefit from Agent Skills, reusable procedural guidance for reasoning, tool use, and task execution (Anthropic, 2025; Li et al., 2026). Because this guidance can be revised without updating model parameters, skill evolution seeks to improve a skill over repeated task executions (Wang et al., 2026b; Yang et al., 2026). When one skill is reused across heterogeneous tasks, however, its revisions

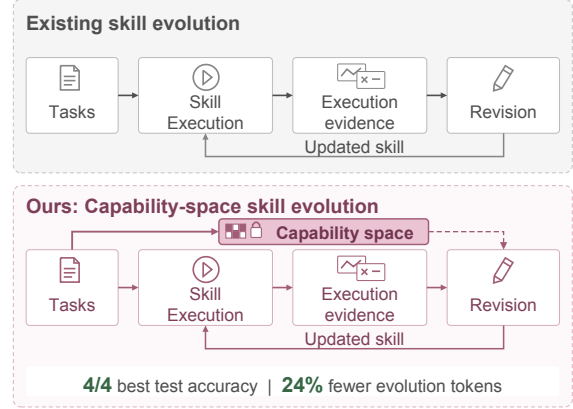


Figure 1: **Existing and capability-space skill evolution.** Both follow the same execution–revision loop. Existing methods base revisions on current execution evidence, whereas ours also derives a capability space from tasks before execution and uses it during revision.

must serve tasks that require different and often overlapping procedural requirements. The practical challenge is therefore to improve one shared skill across heterogeneous tasks without treating every task as a separate revision problem.

Existing skill-evolution methods derive revisions mainly from execution evidence: distilled trajectories (Ni et al., 2026), diagnosed failures (Wang et al., 2026a; Liu et al., 2026b), or scored rollouts (Yang et al., 2026). This evidence shows what the current skill did, but it changes as the skill changes and does not reveal which procedural requirements recur across tasks. Using tasks themselves as revision units does not solve the problem either: one task may require several procedures, while the same procedure may appear in many different tasks. Figure 1 illustrates this gap: execution evidence remains necessary for revision, yet it cannot tell which requirements the tasks share.

A useful unit for skill revision should be finer than a task, yet remain meaningful as the skill changes. Two spreadsheet tasks with different goals, for instance, may both require checking the

*Corresponding author.

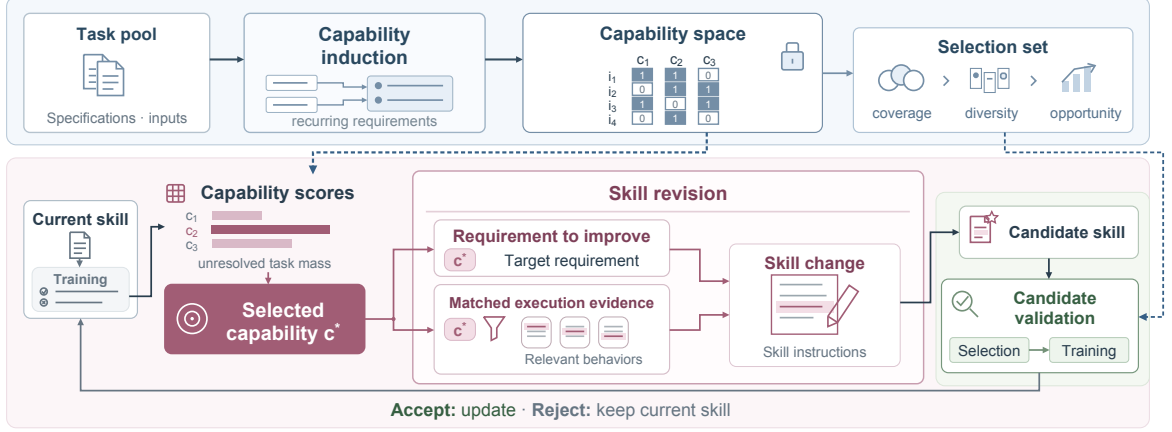


Figure 2: **Overview of SkillFocus.** Before evolution, capability induction builds a fixed capability space (C, Q) from task specifications, from which the selection set is built. In each round, capability scores select c^* , which sets the requirement to improve and the matched execution evidence; the resulting candidate replaces the current skill only after selection and training validation.

populated range before writing, while one of them also requires formula construction and output verification. We call such a recurring procedural requirement a capability when the skill can be revised to handle it better. Revising the skill can improve how well it handles a requirement without changing what the task requires. This leads to our central question: can recurring requirements derived before execution guide both what to revise and which execution evidence to use as the skill changes?

We introduce SkillFocus, which identifies recurring procedural requirements from task specifications before any execution and uses them to define a capability space that remains fixed as the skill changes (Figure 2). After each execution round, SkillFocus maps task outcomes to the capabilities and selects the capability with the most unresolved supporting tasks. The selected capability determines what part of the skill to revise and which execution evidence to use for diagnosis and editing. The resulting candidate is retained only after whole-skill validation.

Across four benchmarks spanning heterogeneous tasks, SkillFocus achieves the best held-out accuracy on all four, outperforming the strongest competing result by 5.7 points on average while using 24% fewer evolution tokens on average than the closest iterative baseline. Controlled comparisons show that the gain depends on the capability space itself: task-semantic and execution-derived alternatives perform worse, and randomizing task-capability assignments reduces final accuracy by up to 20.2 points. Within fixed revision states, evidence matched to the selected capability also pro-

duces stronger candidates. Both the capability decomposition and its use during revision contribute to the final gains.

Our contributions are as follows:

- We formulate skill evolution over a fixed capability space derived from recurring procedural requirements and reuse the same capabilities across revision rounds.
- We introduce a revision process that selects a capability from current task outcomes and uses it to determine both what to revise and which execution evidence to use.
- We show that SkillFocus improves final skill quality and evolution efficiency across heterogeneous tasks, with controlled studies supporting both the capability decomposition and capability-matched evidence.

2 Related Work

Experience Reuse and Procedural Skills.

Experience-driven agents reuse interaction outcomes to improve later task execution (Zhang et al., 2025). Early work folds environment interaction, memory, and tool use into the execution loop itself (Yao et al., 2023; Sumers et al., 2024; Schick et al., 2023), while reflection-based methods carry verbal feedback or distilled lessons into subsequent attempts (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024; Wu et al., 2026). Other work keeps the same experience for later runs as executable skill libraries (Wang et al., 2024, 2026b), reusable workflows (Wang et al., 2025), procedural memory (Mi et al., 2026), or reasoning memories

distilled from successes and failures (Ouyang et al., 2026); memory systems likewise store and retrieve past experience on demand (Xu et al., 2025; Kang et al., 2025; Yu et al., 2026; Xiong et al., 2026).

Skill Evolution from Execution Evidence.

Building on this idea, recent skill-evolution methods repeatedly propose changes to a skill using evidence produced during task execution. Trace2Skill distills lessons from successful and failed trajectories into a transferable skill (Ni et al., 2026), SkillGrad turns trajectory diagnoses into textual update directions (Wang et al., 2026a), SkillOpt revises a retained skill from scored rollouts (Yang et al., 2026), which SkillOpt-Lite trims to a minimal loop validated on held-out samples (Shen et al., 2026), and SkillRevise applies trace-conditioned, execution-anchored edits (Liu et al., 2026b). Others repair recurring failures in batch, keep a change only after replay validation, or evolve the agent’s policy instead of a textual skill (Liu et al., 2026a; Chen et al., 2026; He and Yang, 2026; Zhang et al., 2024). Execution evidence is useful for diagnosing current failures, but it changes as the skill changes, whereas the procedural requirements imposed by the tasks do not. SkillFocus keeps these recurring requirements fixed across rounds and uses current execution evidence to decide which requirement to revise.

Text and Context Optimization. A broader line of work optimizes prompts, program instructions, or persistent context in text space (Ramnath et al., 2025). Instruction optimizers propose and score candidates with an LLM (Yang et al., 2024; Zhou et al., 2023b), search discrete tokens or follow textual gradients (Shin et al., 2020; Pryzant et al., 2023), or evolve and select populations of prompts (Fernando et al., 2024; Do et al., 2024; Agrawal et al., 2026); program optimizers jointly tune instructions and demonstrations across pipeline stages (Khatab et al., 2024; Opsahl-Ong et al., 2024; Chen et al., 2024); and gradient-style and context methods propagate language feedback through text variables or update a persistent play-book (Yuksekgonul et al., 2025; Zhang et al., 2026; Ye et al., 2026). These methods study how to improve mutable text given feedback. The setting studied here asks which recurring task requirement should guide each round of change to one shared skill.

3 Method

We use skill evolution for the full multi-round process of execution, revision, validation, and update. A skill revision is a proposed change to the current skill within one round; it yields a candidate skill, and a skill update replaces the current skill with a candidate that passes validation. SkillFocus first defines the capabilities that remain fixed during evolution (§3.2), then uses the changing task outcomes to choose a capability and propose a revision (§3.3), and finally decides whether the candidate replaces the current skill (§3.4). Figure 2 summarizes the design.

3.1 Problem Formulation

We study shared-skill evolution, where one mutable skill is reused across a task pool. Let A denote the fixed non-skill components of the target agent, including the model, tools, and execution environment. Let $\mathcal{S}$ be the space of valid skills and $S_t \in \mathcal{S}$ the current skill at round t . Let $\mathcal{D} = \{\tau_i\}_{i=1}^N$ denote the task pool. For each task τ_i , $Y_i(S) := Y(\tau_i, S) \in [0, 1]$ denotes its normalized evaluation score when A executes the task with skill S , with larger values indicating better outcomes. Starting from an initial skill S_0 , which may be empty, round t may propose a candidate S'_t , and validation determines the next skill:

$$S_{t+1} = \begin{cases} S'_t, & \text{if } S'_t \text{ is accepted,} \\ S_t, & \text{otherwise.} \end{cases} \quad (1)$$

skill evolution runs for at most $T_{\max}$ rounds and aims to improve task performance. Across rounds, the skill may change, while the task pool and all non-skill components of the agent remain fixed.

3.2 Capability Space

Before evolution, SkillFocus identifies procedural requirements from each task’s specification, input structure, and environment contract, without using any execution outcome. We call a recurring procedural requirement a capability when revising the skill can improve how well it is handled. Requirements from different tasks are merged when the same reusable skill guidance can address them. Tasks that look similar need not share the same procedural requirement, whereas tasks assigned to the same capability do so by construction. Both conditions are needed. Reading requirements from task specifications prevents the capabilities from changing with the current skill, while requiring

shared skill guidance means that revising the skill can improve how those requirements are handled.

Let $\mathcal{C} = \{c_d\}_{d=1}^K$ be the induced capabilities and $Q \in \{0, 1\}^{N \times K}$ the task–capability Matrix (Junker and Sijtsma, 2001; Wang et al., 2020), with

$$Q_{i,d} = \begin{cases} 1, & \tau_i \text{ requires } c_d, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

The entries record requirements, not outcomes. A task may require several capabilities, and one capability may be shared by many tasks. We call the pair $(\mathcal{C}, Q)$ the *capability space*. The tasks define the requirements; revising the skill changes how well they are handled. We fix $\mathcal{C}$ and Q before evolution begins: task outcomes may change after each accepted skill update, while $\mathcal{C}$ and Q do not. Keeping the same capabilities across rounds lets successive skills be compared against the same task requirements. Appendix C details the extraction, consolidation, assignment, and support filtering procedures.

3.3 Capability-Guided Revision

Before the first round, the task pool is split into a training set D_{tr} , whose executions supply revision evidence, and a selection set D_{sel} , which during evolution is used only to check candidates (§3.4). In each round, SkillFocus uses current training outcomes to choose one capability, and then uses that capability to choose the evidence for revising the skill.

Choosing the Next Capability. Let O_t be the execution record of S_t on D_{tr} , and write $y_i^t := Y_i(S_t)$. Each capability has a support share $\pi_d = |\mathcal{D}|^{-1} \sum_{\tau_i \in \mathcal{D}} Q_{i,d}$, fixed with the capability space, and a current score on the training tasks that require it:

$$q_d^t = \frac{\sum_{\tau_i \in D_{\text{tr}}} Q_{i,d} y_i^t}{\sum_{\tau_i \in D_{\text{tr}}} Q_{i,d}}. \quad (3)$$

We collect these as the capability scores $\mathbf{q}^t = (q_d^t)_{d=1}^K$. Neither coverage nor current score is sufficient alone. A rare capability may be poorly handled but affect few tasks, while a common capability may already be well handled. SkillFocus therefore prioritizes their product, the unresolved task mass

$$m_d^t = \pi_d (1 - q_d^t), \quad (4)$$

and selects

$$d_t^* = \arg \max_{d: c_d \in \mathcal{U}_t} m_d^t, \quad c_t^* = c_{d_t^*}. \quad (5)$$

The eligible capability set $\mathcal{U}_t$ contains the capabilities that a round can both act on and check: each has at least one diagnosable failure in D_{tr} , meaning a training execution that fails for a task–logic reason and not an environment fault (Appendix C.2), and at least one supporting task in D_{sel} .

Matching Evidence to the Capability. The selected capability decides which execution evidence the revision sees. A trace records everything the agent did, and most of it is unrelated to any single requirement. Let $\mathcal{H}_t$ contain the revision attempts and outcomes from rounds before t . SkillFocus keeps only the training tasks with $Q_{i,d_t^*} = 1$ and assembles an evidence bundle $\mathcal{E}_t$ from them: executions that show how the requirement was handled, the skill content related to c_t^* , and the entries of $\mathcal{H}_t$ associated with c_t^* . Successful executions are set beside the failures whenever the record contains any. The round thus diagnoses one requirement from the executions that bear on it, instead of from all executions pooled together.

Proposing the Revision. Diagnosis turns this evidence into a skill revision. SkillFocus proposes a revision only when the evidence links the selected requirement, the observed executions, and editable skill content. The editor then adds, replaces, or removes skill content. Otherwise, SkillFocus abstains on that capability and considers the next one in $\mathcal{U}_t$ within the same round. Edits that copy task-specific entities are discarded before execution.

3.4 Candidate Validation

Selection-Set Construction. The selection set is built once, before the first round, by greedy selection over the task pool. Its priorities are ordered: capability coverage first, measured by the same support share π_d that weights unresolved task mass in Equation 4; then diversity over predefined task features, namely task family, input structure, and environment contract labels; then improvement opportunity, measured by the share of a task’s initial executions that fail for diagnosable reasons. A reserve of tasks solved by S_0 keeps regressions visible, and at least one supporting task for every capability remains in the training set, so revision evidence remains available for that capability. The remaining tasks form the training set. Both sets remain fixed after construction. The selection set determines which capabilities have checking tasks and so can enter $\mathcal{U}_t$, while the capability selected in a later round changes neither set. Algorithm C.2

Algorithm 1 SkillFocus

Require: Fixed agent components A , initial skill S_0 , task pool $\mathcal{D}$, selection size M , horizon $T_{\max}$
Ensure: Final skill S

- 1: Induce $(\mathcal{C}, Q)$ from task specifications, inputs, and environment contracts
- 2: $S \leftarrow S_0, \mathcal{H} \leftarrow \emptyset$
- 3: Execute A with S on $\mathcal{D}$ to obtain O_0
- 4: $(D_{\text{sel}}, D_{\text{tr}}) \leftarrow \text{CONSTRUCTSELECTION}(\mathcal{C}, Q, O_0, M)$
- 5: $O \leftarrow O_0$ restricted to D_{tr}
- 6: Compute capability scores $\mathbf{q}$ from O and Q
- 7: **for** $t = 0$ to $T_{\max} - 1$ **do**
- 8: $\mathcal{U} \leftarrow \mathcal{U}_t$ from $\mathbf{q}$
- 9: **repeat**
- 10: **if** $\mathcal{U} = \emptyset$ **then**
- 11: **return** S
- 12: **end if**
- 13: $d_t^* \leftarrow \arg \max_{d: c_d \in \mathcal{U}} m_d^t; \mathcal{U} \leftarrow \mathcal{U} \setminus \{c_{d_t^*}\}$
- 14: Collect evidence $\mathcal{E}_t$ for $c_{d_t^*}^*$ from O, S , and $\mathcal{H}$
- 15: Diagnose, then propose S'_t or abstain
- 16: **until** a candidate S'_t is proposed
- 17: Evaluate S'_t against S on D_{sel} , then on D_{tr}
- 18: **if** both paired changes are positive **then**
- 19: $S \leftarrow S'_t$; set O to the training executions of S'_t from validation; recompute $\mathbf{q}$
- 20: **end if**
- 21: Append the revision records and outcome to $\mathcal{H}$
- 22: **end for**
- 23: **return** S

gives the procedure.

Whole-Skill Validation. Because capabilities overlap and each revision edits the shared skill, a revision aimed at one capability can change outcomes on tasks outside it. Checking the candidate only on the tasks of that capability could accept revisions that repair one requirement while degrading others. SkillFocus evaluates every candidate as a whole instead. For $X \in \{D_{\text{sel}}, D_{\text{tr}}\}$, the paired change is

$$\Delta_X(S', S) := \frac{1}{|X|} \sum_{\tau_i \in X} [Y_i(S') - Y_i(S)]. \quad (6)$$

A candidate is retained only when both paired changes are positive:

$$\text{ACCEPT}(S') \iff \Delta_{D_{\text{sel}}}(S', S) > 0 \quad (7) \\ \wedge \Delta_{D_{\text{tr}}}(S', S) > 0.$$

The two checks serve different roles. selection execution traces are not supplied to candidate generation, so the first check evaluates the candidate on tasks whose execution traces did not inform that revision. The training re-evaluation checks that the candidate produces a positive net change over the training set, which supplies revision evidence across rounds, and it provides the execution record for the next round.

If the candidate is accepted, $S_{t+1} = S'_t$, and its training executions from validation become O_{t+1} , from which the next capability scores $\mathbf{q}^{t+1}$ are computed. Otherwise, $S_{t+1} = S_t$ and $O_{t+1} = O_t$. All capability attempts made in round t , together with the proposed revision and its validation outcome when a candidate is produced, are appended to $\mathcal{H}_{t+1}$. A round ends once a candidate reaches validation, and the loop returns the final skill after the evolution horizon, or earlier when no capability in $\mathcal{U}_t$ yields a candidate. Algorithm 1 summarizes the loop.

4 Experiments

We organize our evaluation around three research questions (RQs):

- **Research Question 1 (RQ1):** Does SkillFocus improve final skill quality and evolution efficiency? (§4.2)
- **Research Question 2 (RQ2):** Does deriving capabilities from recurring task requirements improve skill evolution compared with task-semantic and execution-derived alternatives? (§4.3)
- **Research Question 3 (RQ3):** Does selecting what to revise and which evidence to use with the same capability improve candidate quality? (§4.4)

4.1 Experimental Setup

Benchmarks. We evaluate SkillFocus on four heterogeneous benchmarks. SearchQA provides search-oriented question answering tasks (Dunn et al., 2017). SpreadsheetBench (SSB) evaluates manipulation of real-world workbooks (Ma et al., 2024). LiveMathematicianBench (LiveMath) poses research-level mathematical reasoning as multiple-choice questions drawn from recent papers (He et al., 2026). IFBench measures generalization to verifiable instruction constraints (Pyatkin et al., 2025), extending the verifiable constraints of IFEval (Zhou et al., 2023a). Each benchmark evolves its own shared skill, testing the same method across distinct task types.

Baselines. We compare against five baselines. No Skill runs the target model without procedural guidance; One-shot Skill writes a skill once without execution feedback; Trace2Skill (Ni et al., 2026) distills a skill from execution trajectories; Genetic-Pareto (GEPA) (Agrawal et al., 2026) mutates the

Table 1: **Test accuracy (%) of SkillFocus and baselines on four benchmarks.** Bold and underlined values denote the best and second-best results, respectively; Avg.: unweighted mean over the four benchmarks.

Method	SearchQA	SpreadsheetBench	LiveMath	IFBench	Avg.
No Skill	64.5	21.8	14.4	67.8	42.1
One-shot Skill	70.7	34.0	30.6	<u>80.6</u>	54.0
Trace2Skill	73.7	30.7	39.5	76.3	55.1
GEPA	74.6	58.9	37.9	73.4	61.2
SkillOpt	<u>76.1</u>	<u>62.1</u>	<u>41.1</u>	73.9	<u>63.3</u>
SkillFocus	80.1	77.1	43.6	82.0	70.7

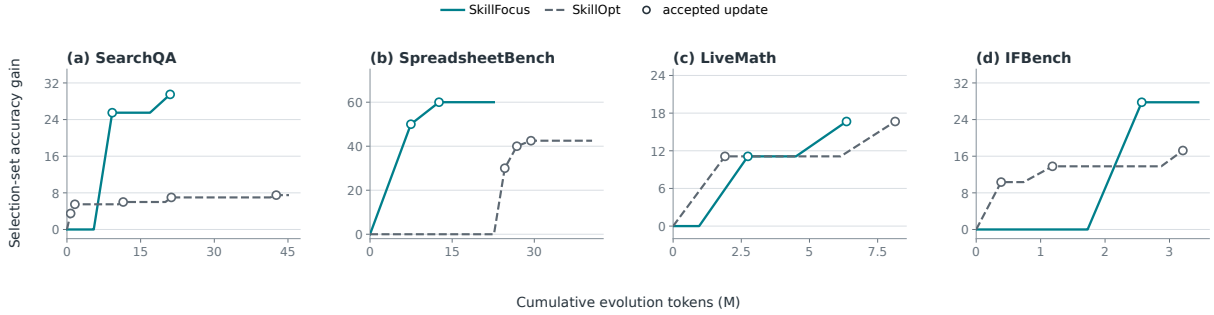


Figure 3: **Evolution trajectories of SkillFocus and SkillOpt.** Each curve plots selection set accuracy gain over the initial skill in percentage points; markers show accepted updates, and horizontal segments are rejected candidates that consume tokens without changing the retained skill.

skill text through reflection and keeps candidates on a Pareto front; and SkillOpt (Yang et al., 2026) iteratively revises a retained skill from rollout evidence.

Protocol. Within each benchmark, all methods use the same evaluator, task pool, held-out test split, and DeepSeek-V4-Flash target model (DeepSeek-AI, 2026); the skill-optimization methods also share the initial skill and use the Generative Pre-trained Transformer model GPT-5.4 (OpenAI, 2026) for optimizer-side calls. Each method follows its own candidate-selection procedure within the same non-test pool, and no method receives additional tasks.

Metrics. We report held-out test accuracy and cumulative evolution tokens, measuring final skill quality and optimization cost, respectively. Evolution cost counts all inference tokens spent before the final skill is fixed; for SkillFocus, this includes capability induction and the initial executions used before iterative revision. The final test evaluation is not counted. Appendix B gives the full protocol, including data splits, optimization budgets, execution settings, and cost accounting.

4.2 Main Results

SkillFocus achieves the highest held-out test accuracy on all four benchmarks (Table 1). Its mar-

gin over the strongest competing result on each benchmark ranges from 1.4 to 15.0 points and averages 5.7 points, with the widest margin over SkillOpt, the closest iterative baseline, on SpreadsheetBench. On IFBench, SkillFocus is the only iterative method that exceeds One-shot Skill, reaching 82.0 against 80.6. Sections 4.3 and 4.4 examine which parts of capability-guided revision produce this advantage.

SkillFocus reaches these accuracies while using 24% fewer evolution tokens than SkillOpt on average, with the largest reduction on SearchQA (54.3%). Figure 3 shows that it reaches larger gains through fewer accepted revisions and, on the first three benchmarks, makes its last accepted revision earlier in the token budget. This is consistent with prioritizing the capability that leaves the most tasks unresolved, so one accepted revision can move many tasks at once. Replacing the constructed selection set with a uniform random sample of the same size also lowers final test accuracy by 3.9–17.2 points (Appendix A.6). Taken together, SkillFocus produces a better final skill on every benchmark while using fewer evolution tokens than SkillOpt on average.

4.3 Analysis of the Capability Space

Tasks often require several capabilities, while different capabilities cover different subsets of tasks.

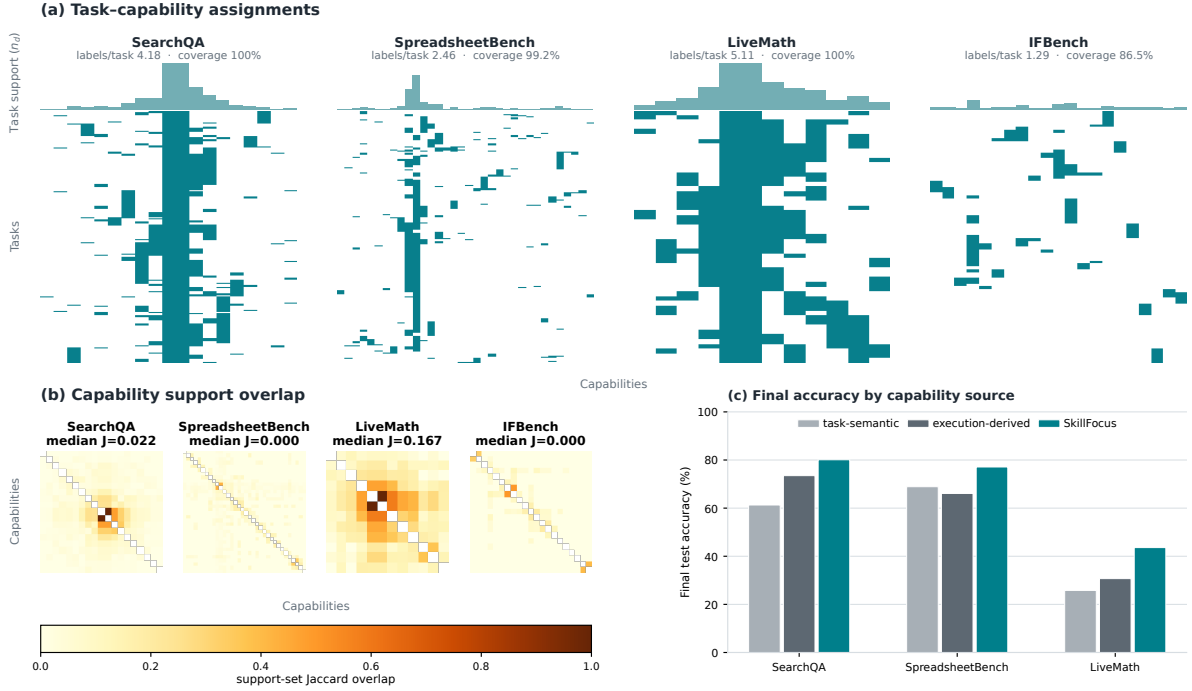


Figure 4: **Task-capability assignments and their effect on final skill accuracy.** (a) capabilities assigned to each task and the number of tasks per capability; (b) overlap between the task sets of different capabilities; (c) final skill accuracy when capabilities are derived from different sources. Panel annotations report labels per task, coverage, and median support overlap.

On SearchQA, SpreadsheetBench, and LiveMath, at least 86% of tasks require multiple capabilities, and 36% do so on IFBench (Figure 4a). The median Jaccard overlap between the task sets of two capabilities stays at or below 0.17 on every benchmark (Figure 4b).

Where capabilities come from changes the quality of the evolved skill. Task-semantic grouping uses overall task similarity, whereas execution-derived grouping uses the outcomes of the initial skill. With the rest of the pipeline fixed, capabilities derived from procedural requirements outperform the former by 8.2–18.8 points and the latter by 6.6–12.9 points on all three evaluated benchmarks (Figure 4c), and which alternative comes second varies by benchmark. Unlike the execution-derived grouping, these requirements are defined by the tasks and stay the same as the skill changes.

Which tasks share a capability matters. The randomized control preserves how many capabilities each task has and how many tasks each capability covers, but breaks which tasks are linked to which requirements, and final accuracy drops by 11.1–20.2 points (Table 5). The gain therefore comes from the exact task-capability assignments, not only from the number or size of the capability groups.

Independent proposal views also recover simi-

lar task-capability assignments on all four benchmarks, with an adjusted Rand index (ARI) of 0.883–0.973 and normalized mutual information (NMI) of 0.953–0.989 (Appendix A.2). Together, these results show that deriving capabilities from recurring task requirements yields better final skills than the task-semantic and execution-derived alternatives, and that the specific task-capability assignments are both consequential and reproducible.

4.4 Analysis of Capability-Guided Revision

We next test whether candidates improve when the revision target and its evidence refer to the same capability. We hold 15 historical revision states fixed and rerun candidate generation under a 2×2 crossover: the selected capability is either the highest-priority or a random non-top capability, and the evidence is either matched or mismatched to that capability (Table 2). Each condition thus starts from the same current skill and execution history; only the chosen capability and the evidence supplied to revision change.

Candidates are strongest when SkillFocus revises the highest-priority capability using evidence from tasks that require that same capability. We compare candidates by the training gain Δ_{tr} at the same historical state, since the training executions supply the revision evidence. Matched evidence

Table 2: **Focus–evidence** crossover over 15 historical revision states, macro-averaged across SearchQA, SpreadsheetBench, and LiveMath. Accept: candidate acceptance rate; Δ_{sel} , Δ_{tr} : paired candidate gains (points) on the selection and training sets.

Condition	Accept (%)	Δ_{sel}	Δ_{tr}
TM: Top + Matched	40.0	+4.10	+9.37
T↔M: Top + Mismatched	26.7	+3.62	+4.97
RM: Random + Matched	26.7	+2.28	+5.33
R↔M: Random + Mismatched	20.0	+2.09	+3.39

raises Δ_{tr} for both the highest-priority and a random capability, and top-priority selection raises it under both matched and mismatched evidence. Under top-priority selection, matched evidence adds 4.4 points; Top + Matched reaches the highest Δ_{tr} (+9.37) and the highest acceptance rate (40.0%), so stronger candidates also pass validation more often.

Whole-run ablations show the same pattern: removing capability-matched evidence, or removing both matched evidence and capability prioritization, lowers final skill accuracy on all three evaluated benchmarks (Appendix A.3). The crossover isolates candidate quality from the same starting state, while the whole-run ablations show that these per-round gains reach the final skill. Using one capability to choose both what to revise and which evidence to read therefore improves the candidates a round produces, and those gains carry through to the final skill.

5 Conclusion

We presented SkillFocus, a framework for evolving a shared Agent skill through a fixed capability space derived from recurring procedural requirements. As the skill changes, SkillFocus uses current task outcomes to choose which capability to improve, and uses that same capability to select the execution evidence for revision. Across heterogeneous benchmarks, SkillFocus produces stronger final skills at a lower average evolution cost, and controlled studies support both the capability space and its use to guide revision. More broadly, skill evolution can separate what tasks repeatedly require from how the current skill behaves, keeping the former fixed and using the latter to decide what to improve and which evidence to revise from.

Limitations

SkillFocus assumes that recurring procedural requirements can be identified before evolution and

that these requirements are addressable through skill revision. Because each revision changes the shared skill as a whole, improving one capability can also change behavior on other requirements; SkillFocus validates every candidate as a whole rather than confining an edit to its target. Our evaluation studies bounded evolution under one optimizer–target configuration; broader model families and longer-horizon or cross-domain evolution remain future directions.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. [Gepa: Reflective prompt evolution can outperform reinforcement learning](#). In *International Conference on Learning Representations*.
- Anthropic. 2025. [Equipping agents for the real world with agent skills](#). Anthropic Engineering.
- Kunfeng Chen, Qihuang Zhong, Juhua Liu, and Bo Du. 2026. [Skillcat: Contrastive, assessment-augmented and topology-aware skill self-evolution for llm agents](#). *Preprint*, arXiv:2606.13317.
- Yongchao Chen, Jacob Arkin, Yilun Hao, Yang Zhang, Nicholas Roy, and Chuchu Fan. 2024. [PRompt optimization in multi-step tasks \(PROMST\): Integrating human feedback and heuristic-based sampling](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3859–3920. Association for Computational Linguistics.
- DeepSeek-AI. 2026. [DeepSeek-V4: Towards highly efficient million-token context intelligence](#). *Preprint*, arXiv:2606.19348.
- Xuan Long Do, Yiran Zhao, Hannah Brown, Yuxi Xie, James Xu Zhao, Nancy F. Chen, Kenji Kawaguchi, Michael Shieh, and Junxian He. 2024. [Prompt optimization via adversarial in-context learning](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7308–7327. Association for Computational Linguistics.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. [Improving factuality and reasoning in language models through multiagent debate](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763. PMLR.

- Matthew Dunn, Levent Sagun, Mike Higgins, V. Ugur Guney, Volkan Cirik, and Kyunghyun Cho. 2017. [Searchqa: A new q&a dataset augmented with context from a search engine](#). *Preprint*, arXiv:1704.05179.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. [Promptbreeder: Self-referential self-improvement via prompt evolution](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 13481–13544. PMLR.
- Linyang He, Qiyao Yu, Hanze Dong, Baohao Liao, Xinxing Xu, Micah Goldblum, Jiang Bian, and Nima Mesgarani. 2026. [Livemathematicianbench: A live benchmark for mathematician-level reasoning with proof sketches](#). In *Conference on Language Modeling (COLM)*.
- Yu He and Weikai Yang. 2026. [Skillcommit: Evolving agent skills through behaviorally validated scope expansion](#). *Preprint*, arXiv:2608.15165.
- Brian W. Junker and Klaas Sijtsma. 2001. [Cognitive assessment models with few assumptions, and connections with nonparametric item response theory](#). *Applied Psychological Measurement*, 25(3):258–272.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. [Memory os of ai agent](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25961–25970. Association for Computational Linguistics.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. [Dspy: Compiling declarative language model calls into self-improving pipelines](#). In *International Conference on Learning Representations*.
- Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. [Skills-bench: Benchmarking how well agent skills work across diverse tasks](#). *Preprint*, arXiv:2602.12670.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. 2026a. [Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support](#). In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 4763–4768. Association for Computing Machinery. Also available as arXiv:2604.08618.
- Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyang Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Chenyu Ruan, Haoran Li, and Yangqiu Song. 2026b. [Skillre-vise: Improving llm-authored agent skills via trace-conditioned skill revision](#). In *Findings of the Association for Computational Linguistics: EMNLP 2026*.
- Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. 2024. [Spreadsheetbench: Towards challenging real world spreadsheet manipulation](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). In *Advances in Neural Information Processing Systems*, volume 36.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. [Procmem: Learning reusable procedural memory from experience via non-parametric ppo for llm agents](#). In *Proceedings of the 43rd International Conference on Machine Learning*. Also available as arXiv:2602.01869.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjuan Jiang. 2026. [Trace2skill: Distill trajectory-local lessons into transferable agent skills](#). *Preprint*, arXiv:2603.25158.
- OpenAI. 2026. [GPT-5.4 thinking system card](#).
- Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. [Optimizing instructions and demonstrations for multi-stage language model programs](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366. Association for Computational Linguistics.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. [Reasoningbank: Scaling agent self-evolving with reasoning memory](#). In *International Conference on Learning Representations*.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. [Automatic prompt optimization with “gradient descent” and beam search](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968. Association for Computational Linguistics.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. 2025. [Generalizing verifiable instruction following](#). In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 38.

- Kiran Ramnath, Kang Zhou, Sheng Guan, Soumya Smruti Mishra, Xuan Qi, Zhengyuan Shen, Shuai Wang, Sangmin Woo, Sullam Jeoung, Yawei Wang, Haozhu Wang, Han Ding, Yuzhe Lu, Zhichao Xu, Yun Zhou, Balasubramaniam Srinivasan, Qiaojing Yan, Yueyan Chen, Haibo Ding, and Lin Lee Cheong. 2025. [A systematic survey of automatic prompt optimization techniques](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33078–33110. Association for Computational Linguistics.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Advances in Neural Information Processing Systems*, volume 36.
- Yifei Shen, Bo Li, and Xinjie Zhang. 2026. [Skillopt-lite: Better and faster agent self-evolution via one line of vibe](#). *Preprint*, arXiv:2607.03451.
- Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, and Sameer Singh. 2020. [Autoprompt: Eliciting knowledge from language models with automatically generated prompts](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 4222–4235. Association for Computational Linguistics.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). In *Advances in Neural Information Processing Systems*, volume 36.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. [Cognitive architectures for language agents](#). *Transactions on Machine Learning Research*.
- Fei Wang, Qi Liu, Enhong Chen, Zhenya Huang, Yuying Chen, Yu Yin, Zai Huang, and Shijin Wang. 2020. [Neural cognitive diagnosis for intelligent education systems](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 6153–6161. AAAI Press.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. [Voyager: An open-ended embodied agent with large language models](#). *Transactions on Machine Learning Research*.
- Hanyu Wang, Yifan Lan, Bochuan Cao, Lu Lin, and Jinghui Chen. 2026a. [Skillgrad: Optimizing agent skills like gradient descent](#). *Preprint*, arXiv:2605.27760.
- Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. 2026b. [Reinforcement learning for self-improving agent with skill library](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1529–1550. Association for Computational Linguistics.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). In *International Conference on Learning Representations*.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. [Agent workflow memory](#). In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2026. [From interactions to principles: Experience-driven self-distillation for evolving llm agents](#). In *Proceedings of the 43rd International Conference on Machine Learning*. Also available as arXiv:2510.16079.
- Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2026. [How memory management impacts llm agents: An empirical study of experience-following behavior](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645. Association for Computational Linguistics.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-mem: Agentic memory for llm agents](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 20004–20031. Neural Information Processing Systems Foundation.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. [Large language models as optimizers](#). In *International Conference on Learning Representations*.
- Yifan Yang, Ziyang Gong, Wei-quan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. 2026. [Skillopt: Executive strategy for self-evolving agent skills](#). *Preprint*, arXiv:2605.23904.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations*.
- Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. 2026. [Meta context engineering via agentic skill evolution](#). *Preprint*, arXiv:2601.21557.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. [Agentic memory: Learning unified long-term and short-term memory management for large language model](#)

- agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21457–21483. Association for Computational Linguistics.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. [Optimizing generative ai by backpropagating language model feedback](#). *Nature*, 639(8055):609–616.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. [Agentic context engineering: Evolving contexts for self-improving language models](#). In *International Conference on Learning Representations*.
- Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. 2024. [Agent-pro: Learning to evolve via policy-level reflection and optimization](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5348–5375. Association for Computational Linguistics.
- Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. [A survey on the memory mechanism of large language model-based agents](#). *ACM Transactions on Information Systems*, 43(6):155:1–155:47.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. [Expel: Llm agents are experiential learners](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023a. [Instruction-following evaluation for large language models](#). *Preprint*, arXiv:2311.07911.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023b. [Large language models are human-level prompt engineers](#). In *International Conference on Learning Representations*.

Appendix Overview

Appendix A expands the evidence for the two central claims of the paper: the capability space groups tasks in a structured and reproducible way that changes the evolved skill, and capability-guided revision improves candidate quality. Appendix B documents the evaluation protocol, fairness controls, and cost accounting. Appendix C details each stage of the method with its pseudocode, Appendix D gives representative prompt interfaces, and Appendix E states how large language models are used.

A Extended Analysis

Beyond the two central claims, this section examines how far a whole-skill revision reaches beyond its target tasks, how selection accuracy relates to held-out accuracy, where the evolution budget goes, and one complete revision chain.

A.1 Capability Structure and Functional Relevance

No fixed human taxonomy covers the induced capabilities, so we evaluate them by their structure, their stability across proposal views, and the skill they produce, instead of by a ground-truth clustering score.

Structure. A task usually requires several capabilities, and each capability covers a different number of tasks. The mean number of active capabilities per task ranges from 1.29 on IFBench to 5.11 on LiveMath, while support ranges from small task subsets to requirements shared by the whole pool (Table 3). Figure 4 shows the same three facts as panels: which capabilities each task carries, how far their task sets overlap, and what the resulting skill scores.

Table 4 gives representative behaviors grouped into capabilities. SpreadsheetBench combines a high-support formula-writing behavior with narrower matching and reference behaviors, while IFBench contains sparse requirements attached to smaller task subsets. A task may carry several capabilities at once.

Functional relevance. Table 5 tests three alternative explanations for the benefit of the capability space. Each alternative replaces one property of the frozen (C, Q) while the downstream evolution pipeline stays fixed (Appendix B.5.3).

Not generic task grouping. The task-semantic decomposition groups tasks by their overall objective, operation, and domain, reading the same task specifications and allowed the same number of capabilities. It trails SkillFocus by 18.8, 8.2, and 17.8 points on SearchQA, SpreadsheetBench, and LiveMath. Tasks with similar surface semantics may share no behavior that one skill edit can change, and a task-semantic label keeps a task whole instead of splitting it into the separate requirements a revision can address.

Not execution-derived grouping. The execution-derived decomposition groups tasks by neutral summaries of how the initial skill behaved on the training set. It is the stronger alternative on SearchQA and LiveMath, yet it still trails by 6.6, 11.0, and 12.9 points. Its groups describe how the initial skill behaved, so they keep pointing at that skill while later rounds change it, which is consistent with its lower endpoints.

Not matrix statistics alone. The randomized capability assignment preserves the task and capability degree sequences of Q while breaking which task carries which requirement. It reduces accuracy by 11.1, 17.5, and 20.2 points, so which task carries which requirement, and not the degree sequences alone, drives the gain.

Across the three controls, capabilities read from task specifications give the strongest final skill on every benchmark, and they are the only source that stays fixed while the skill changes.

A.2 Assignment Stability

We assess stability across three independent proposal calls that read the same task specifications and do not observe one another’s outputs. After aligning common behavioral-requirement references, we compare the resulting assignments using the adjusted Rand index (ARI) and normalized mutual information (NMI).

The three views propose different numbers of capabilities, yet they assign tasks to them in much the same way on all four benchmarks (ARI .883–.973; NMI .953–.989; Table 6), so the assignments survive a change of proposal granularity and rest on no single view.

A.3 Capability-Guided Revision: Full Crossover Analysis

Choosing the capability to revise and the evidence to read from the same capability produces the strongest candidates. The crossover shows this

Table 3: Benchmark-level statistics of the frozen task–capability matrices. Tasks is the number of task records the matrix covers; K is the number of induced capabilities; Labels per task is the mean number of active capabilities per task; Support is the minimum–maximum number of supporting tasks; J is pairwise support-set Jaccard overlap; and Coverage is the fraction of covered tasks carrying at least one capability.

Benchmark	Tasks	K	Labels per task	Support	Mean J	Median J	Coverage
SearchQA	600	19	4.18	3–600	.055	.022	100%
SpreadsheetBench	120	34	2.46	3–89	.018	.000	99.2%
LiveMath	53	12	5.11	5–53	.202	.167	100%
IFBench	89	21	1.29	3–18	.018	.000	86.5%

Table 4: Representative behavioral requirements and their task support.

Benchmark	Representative behavioral requirement	# Tasks
SearchQA	Return exactly one non-empty answer element containing only the final answer text, with no reasoning, citations, or extra prose.	600
SearchQA	Use only the supplied question, documents, and metadata as evidence, avoiding outside knowledge, tools, and unsupported inference.	600
SearchQA	Select an answer only when the same candidate satisfies all substantive clue constraints, rather than combining separate partial matches.	191
SearchQA	Follow the relation expressed in the clue to extract the correct participant, such as an author, actor, founder, owner, or creator.	127
SpreadsheetBench	Write the exact formula or formula-based expression required by the task directly in the specified answer cells.	89
SpreadsheetBench	Generate each output with correct row-specific references, conditions, and source matches rather than positional assumptions.	31
SpreadsheetBench	Identify and process rows by comparing the required keys, text, or filter conditions so only matching records contribute.	17
LiveMathematicianBench	Base the answer only on the supplied prompt and choices, avoiding outside knowledge, retrieval, tools, and code execution.	53
LiveMathematicianBench	Select the strongest claim supported by the prompt while rejecting weaker consequences and unsupported overstatements.	31
LiveMathematicianBench	Preserve exact quantifier order, object scope, endpoint cases, and domain restrictions when comparing choices.	22
IFBench	Use the structural conventions of the requested output genre, document form, or presentation format rather than generic prose.	18
IFBench	Satisfy exact, minimum, maximum, distinctness, or repetition counts for specified lexical categories while avoiding unintended extras.	13
IFBench	End every sentence or sentence-like line with an emoji when the instruction requires it.	3

Table 5: Test accuracy (%) when capabilities are derived from different sources, under the same downstream SkillFocus pipeline.

capability source	SearchQA	SSB	LiveMath
SkillFocus	80.1	77.1	43.6
task-semantic	61.3	68.9	25.8
execution-derived	73.5	66.1	30.7
randomized assignment	69.0	59.6	23.4

Table 6: Assignment agreement across three independent proposal calls on the same task specifications. K : proposal-specific capability counts; Common refs.: aligned behavioral-requirement references; ARI and NMI: adjusted Rand index and normalized mutual information between proposal assignments.

Benchmark	Views	K	Common refs.	ARI	NMI
SearchQA	3	18/13/17	47.7	.883	.953
SSB	3	17/16/19	35.7	.913	.976
LiveMath	3	11/11/10	33.3	.973	.989
IFBench	3	30/24/42	112.3	.912	.974

Table 7: Per-benchmark focus–evidence crossover (five historical revision states per benchmark). Accept: candidate acceptance rate; Δ_{sel} , Δ_{tr} : paired candidate gains (points) on the selection and training sets.

Condition	Accept (%)	Δ_{sel}	Δ_{tr}
<i>SearchQA</i>			
TM	40.0	+5.30	+10.50
T→M	20.0	+5.30	+3.42
RM	40.0	+7.50	+3.92
R→M	40.0	+3.13	+1.92
<i>SpreadsheetBench</i>			
TM	40.0	+7.00	+13.33
T→M	20.0	+4.17	+7.71
RM	40.0	+6.00	+8.10
R→M	0.0	+5.00	+4.78
<i>LiveMath</i>			
TM	40.0	0.00	+4.29
T→M	40.0	+1.39	+3.77
RM	0.0	−6.67	+3.98
R→M	20.0	−1.85	+3.47

through two controlled contrasts: matched against

mismatched evidence under a fixed focus policy (TM–T–M and RM–R–M), and top-priority against randomized focus under matched evidence (TM–RM).

Evidence alignment. On the training set, matched evidence raises candidate gain under top-priority focus by 7.08, 5.62, and 0.52 points on SearchQA, SpreadsheetBench, and LiveMath, and under randomized focus by 2.00, 3.32, and 0.51 points (Table 7). The direction holds under both focus policies and on all three benchmarks, so evidence drawn from the capability under revision beats evidence drawn from another one whichever capability the round selects.

Capability prioritization. With matched evidence, top-priority focus exceeds randomized focus by 6.58, 5.23, and 0.31 points, so targeting the capability with the largest unresolved task mass adds to what matching the evidence already gives.

Joint reading. TM, the only condition that prioritizes a capability and reads its own evidence, attains the largest training set candidate gain on every benchmark and a 40.0% acceptance rate on each. Margins are smallest on LiveMath, where selection set contrasts are also less uniform; the training set candidate gain serves as the primary crossover comparison (Appendix B.5.2).

Table 8: Ablations of capability-guided revision on SearchQA, SpreadsheetBench, and LiveMath (test accuracy, %).

Variant	SearchQA	SSB	LiveMath
SkillFocus	80.1	77.1	43.6
w/o matched evidence	71.4	58.6	20.8
w/o capability prioritization	68.7	38.3	20.8

Selected Capabilities in the formal runs. Table 9 records which capabilities the priority rule of Equation 4 actually selected over the five rounds of each formal run. The selected capability is not always the broadest one: SpreadsheetBench repeatedly selects a requirement covering 74% of its task population, LiveMath alternates between full-support and partial-support requirements, and IFBench selects requirements covering between 3% and 20% of its tasks. Selecting a capability narrows the pool of task records only when that capability has partial support. At full support the evidence population stays the same, and the selected capability still states the requirement that frames diagnosis and limits the injected revision history to earlier attempts on that requirement.

Table 9: Capabilities selected by the priority rule over the five rounds of each formal run. Support is the number of tasks requiring the capability in the frozen Q ; Share is its fraction of the task population. A capability is selected in consecutive rounds when it is the only eligible one.

Benchmark	capability	Rounds	Support	Share
SearchQA	c_1	3	600	100.0%
SearchQA	c_2	1	600	100.0%
SearchQA	c_3	1	289	48.2%
SpreadsheetBench	c_1	5	89	74.2%
LiveMath	c_1	2	53	100.0%
LiveMath	c_3	2	31	58.5%
LiveMath	c_7	1	24	45.3%
IFBench	c_{29}	3	3	3.4%
IFBench	c_8	1	18	20.2%
IFBench	c_{12}	1	4	4.5%

End-to-end ablations. Table 8 reports the end-to-end ablations defined in Appendix B.5.1. Under global evidence, keeping capability prioritization raises the endpoints by 2.7, 20.3, and 0.0 points on SearchQA, SpreadsheetBench, and LiveMath; with prioritization kept, matching the evidence to the selected capability adds 8.7, 18.5, and 22.8 points. LiveMath separates the two most sharply: prioritization under global evidence recovers none of the loss, while matched evidence recovers 22.8 points. These ablations rerun the whole evolution trajectory, so they measure what each component contributes to the final skill, alongside the same-state crossover above.

A.4 Behavioral Scope of Capability-Guided Revisions

A whole-skill edit can change tasks beyond the ones that selected its capability. Table 10 measures how far each recorded change concentrates on target tasks.

Table 10: Target and non-target edit responses. T/N: group sizes; Δ_T/Δ_N : net changes (points); $R_d(e)$: their difference; Screen./Acc.: screening record/accepted candidate (Acc.-1: first accepted).

Benchmark	Record	T/N n	Δ_T/Δ_N	$R_d(e)$
SQA	Screen.	93/107	+1.1/ +0.0	+1.1
SSB	Acc.-1	89/31	+46.1/ +16.1	+29.9
LM	Acc.	31/22	+6.5/ +4.5	+1.9
IFB	Acc.	3/86	+0.0/ +8.1	-8.1

The target behaviors in the recorded responses are selecting the answer type requested by a clue, writing the exact formula in specified answer cells, selecting the strongest claim supported by a prompt,

and ending each required sentence with an emoji. Target and non-target groups are defined by the frozen task–capability Matrix. $R_d(e)$ is the target-minus-non-target difference in net percentage-point change.

SpreadsheetBench shows the clearest targeted response (Table 10), consistent with a specific target-cell contract. SearchQA and LiveMath show smaller positive contrasts (+1.1 and +1.9 percentage points), indicating that reusable skill rules can benefit both target and neighboring behaviors. IFBench shows a negative contrast (−8.1 percentage points), with its improvement appearing mainly outside a three-task target group.

Appendix A.7 examines the first accepted SpreadsheetBench edit in detail. The second accepted edit has a positive target response together with a larger non-target response, recording the collateral changes possible when whole-skill instructions affect overlapping behaviors. The IFBench response contains three target tasks and a much larger non-target group; its support size is reported with the response value.

A selected capability therefore decides which evidence a round reads and which direction the edit takes; it does not fence in where the edit lands. Whole-skill screening is what keeps the aggregate change positive.

A.5 Selection Screening and Held-out Transfer

The selection set screens candidates inside the loop, while test accuracy reports what the retained skill transfers to unseen tasks. Table 11 lists both endpoints for SkillFocus and SkillOpt. SkillFocus builds its selection set for capability coverage and remaining headroom, so the set collects tasks the initial skill leaves unresolved, and its accuracy stays below the corresponding test accuracy on three of the four benchmarks: 49.5 against 80.1 on SearchQA, 38.9 against 43.6 on LiveMath, and 77.8 against 82.0 on IFBench. A set built this way is meant to score low.

SkillOpt shows the opposite ordering on SpreadsheetBench and IFBench, where its screening accuracy of 85.0 and 89.7 exceeds its test accuracy of 62.1 and 73.9 by 22.9 and 15.8 points. SkillFocus screens lower on both benchmarks and still reaches the higher test accuracy. A screening score reports how well a candidate fits the tasks it was retained on, and it carries no guarantee about unseen tasks.

The two columns cover different task sets, and

Table 11: Final selection set and held-out test accuracy (%). The two columns cover different task sets, so their difference is not a calibrated generalization gap across methods.

Benchmark	SkillFocus		SkillOpt	
	Sel.	Test	Sel.	Test
SearchQA	49.5	80.1	76.0	76.1
SpreadsheetBench	80.0	77.1	85.0	62.1
LiveMath	38.9	43.6	44.4	41.1
IFBench	77.8	82.0	89.7	73.9

the selection sets differ in composition across methods, so the screening-to-test difference carries no cross-method claim about generalization. What the table does support is how to read the trajectories in Figure 3, which plot selection set gain along the search path, and the construction control below, which measures what a different selection set retains.

A.6 Evolution Cost and Selection-Set Design

SkillFocus spends its evolution budget on diagnosing one requirement at a time and on rejecting weak candidates early, which Figure 3 and the token ledger in Table 15 show from two sides. The curves plot selection set accuracy gain against cumulative evolution tokens instead of round counts: horizontal segments are rejected candidates that consume tokens without changing the retained skill, and jumps follow accepted updates. The ledger separates one-time initialization, iterative evolution, and final test set inference.

The evolution cost can be read as

$$C_{\text{evo}} = C_{\text{base}} + C_{\text{induction}} + C_{\text{revision}} + C_{\text{validation}},$$

where C_{base} is the initial evaluation of the current skill, $C_{\text{induction}}$ is the one-time cost of inducing and freezing ($\mathcal{C}, Q$), C_{revision} covers per-round diagnosis and edit generation, and $C_{\text{validation}}$ covers candidate screening, training set re-evaluation, and recomputation of the capability scores. In Table 15, Initialization aggregates $C_{\text{base}} + C_{\text{induction}}$, while Iterative Evolution aggregates $C_{\text{revision}} + C_{\text{validation}}$. SkillFocus pays $C_{\text{induction}}$ once. In exchange, every round reads the capability scores, the selected capability, and its evidence off Q and the current execution record, and never re-derives which tasks and failures belong together. Screening on the selection set first further limits $C_{\text{validation}}$: a candidate that fails there is rejected without training set re-evaluation, and an accepted candidate’s train-

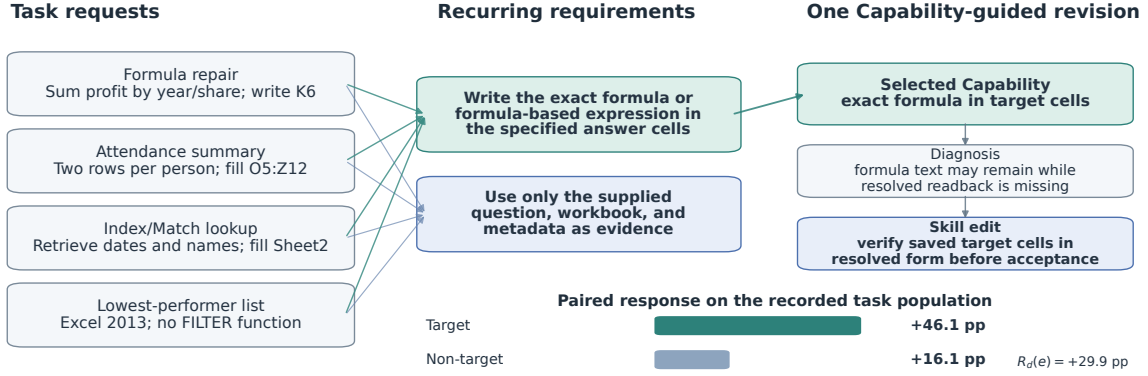


Figure 5: Representative capability-guided revision on SpreadsheetBench. The case connects diverse task requests to a recurring behavioral requirement, an evidence-grounded revision, and paired target/non-target responses.

4. Confirm the target cells/range contain the expected values.

Target contract: verify the requested result in the task-specified target cells or ranges.

4. Confirm the target cells/range by reopening or rereading the saved output and checking the concrete saved values, not only the formula string.

Figure 6: Representative accepted skill revision on SpreadsheetBench. The first excerpt is the current instruction; the second is the accepted candidate’s added target-cell verification and read-back contract.

ing set record becomes the next execution record. The saving comes from where tokens are spent, not from making fewer model calls.

Appendix A.5 shows what the constructed selection set screens on; the control below shows what a different one retains. Replacing the constructed selection set with uniform random sampling of the same size (Appendix B.5.4) lowers the final test accuracy on all three evaluated benchmarks, by 5.1 points on SearchQA, 3.9 points on SpreadsheetBench, and 17.2 points on LiveMath. Which tasks screen the candidates decides which skill survives the loop: tasks chosen for capability coverage, feature diversity, and diagnosable failure retain different candidates than a uniform sample, and every retained candidate changes the skill, the next execution record, and the next capability priority. A selection set built for coverage and headroom keeps more of the updates that transfer beyond the training evidence, and the effect is largest on LiveMath, where the set is smallest. A selection set constructed independently of the priority rule would separate its composition from capability prioritization directly; the uniform random sample is the closest available approximation, and a dedicated control remains open.

A.7 Representative Capability-Guided Revision

We present one complete revision chain from the recorded SpreadsheetBench run, following the same objects SkillFocus works with: a recurring requirement, the evidence matched to it, a diagnosis, a whole-skill revision, and paired candidate responses.

Cross-task Capability and evidence. The four requests in Figure 5 have different surface goals, including formula repair, attendance summarization, lookup construction, and a version-constrained list. They share one requirement: write the exact formula or formula-based expression into the specified answer cells. The frozen Q assigns that requirement to the same capability across the displayed rows, so one revision can be evaluated on tasks that look unrelated.

Evidence-grounded diagnosis.

Observed pattern. Executions under the current skill may write a formula string without verifying the resolved value in the requested target cells.

Diagnosis. The skill lacks an explicit post-save check that reopens the output and verifies the concrete values in the task-specified cells.

Editable location. Extend the target-cell confirmation instruction with an identity check and a read-back check before the task output is considered complete.

The evidence bundle combines failures on the target capability, a successful contrast, related current-skill content, and earlier revisions of the same capability. The diagnosis ties this evidence to an observed execution pattern and to one place in the skill. The resulting edit is applied to the complete skill before candidate screening.

Skill edit. The accepted edit adds a target contract and strengthens the confirmation step of the targeted instruction (Figure 6); the complete skill files are omitted for brevity.

Table 12: Paired response of the representative SpreadsheetBench revision. Repair/reg. counts failure-to-success and success-to-failure changes; the target-minus-non-target contrast is +29.9 points.

Group	n	Repair/reg.	Net Δ
Target	89	42/1	+46.1%
Non-target	31	7/2	+16.1%

Paired candidate response. The case materials thus run from capability-matched evidence to a diagnosis, a whole-skill edit, and a paired candidate evaluation.

Why the Capability matters in this case. The four tasks differ in surface form, yet their failures expose the same target-cell contract. A task-local repair could specialize to one spreadsheet operation, whereas capability-matched evidence pools failures and successes across these tasks before diagnosis. The revision therefore changes a reusable verification rule instead of adding an instance-specific workaround, and it moves target tasks more than non-target ones (Table 12).

B Experimental Details

This section documents how the experiments are constructed; the corresponding observations are reported in Section 4 and Appendix A.

B.1 Benchmarks and Data Split

Table 13 summarizes the four benchmarks. SearchQA tests whether the agent returns the answer type and entity requested by a clue. SpreadsheetBench requires writing correct formulas or values into the specified cells of a workbook. LiveMathematicianBench requires selecting the strongest mathematical claim among shuffled options without code execution. IFBench constrains responses along count, format, ratio, repetition, sentence, word, and custom dimensions. Each bench-

Table 13: Benchmarks used in the experiments. Task format states what the agent reads and produces; Evaluation states the deterministic checker applied to each response.

Benchmark	Task format	Evaluation
SearchQA	Jeopardy!-style clues answered in a closed-context, single-turn setting	Deterministic answer matching
SpreadsheetBench	Spreadsheet manipulation on the Verified subset; the agent reads and writes workbook files	Target cells compared with the reference workbook
LiveMath	Multiple-choice mathematical reasoning with deterministically shuffled options; code execution is disabled	Exact match of the selected option
IFBench	Single-turn free-text responses under verifiable output constraints	Programmatic constraint verification

mark evolves an independent skill from an initial skill that mirrors the SkillOpt setting for that benchmark, all methods share the same evaluator, and every reported score is accuracy on the held-out test set.

The formal protocol uses

$$|D_{\text{tr}}| : |D_{\text{sel}}| : |D_{\text{test}}| = 2 : 1 : 7.$$

Let $\mathcal{D}$ denote the task pool after removing the held-out test set. SkillFocus induces and freezes its capability space over $\mathcal{D}$, executes the initial skill on the same pool to obtain O_0 , and then constructs $(D_{\text{sel}}, D_{\text{tr}})$ from $(\mathcal{C}, Q, O_0)$ as specified in Section 3.4. The construction prioritizes capability coverage, task diversity, and diagnosable improvement opportunity; a reserve of tasks solved by S_0 keeps regressions visible. Both sets are fixed before the first round. selection outcomes enter the loop through candidate screening, and the test set stays closed until final evaluation.

B.2 Baselines and Fairness

No Skill runs the target model on the benchmark task without procedural guidance. *One-shot Skill* applies a single skill document created once before evaluation, without iterative execution feedback. *Trace2Skill* (Ni et al., 2026) executes the target agent on the training set, lets independent optimizer analysts propose local skill patches from individual trajectories, and consolidates the patches into one skill document with a final optimizer call. *GEPA*

(Agrawal et al., 2026) treats the skill text as a single optimizable component and applies reflective mutation with Pareto-based candidate selection, a reflection minibatch of three examples, and a budget of 160 metric calls. *SkillOpt* (Yang et al., 2026) uses its official implementation to iteratively optimize a retained skill. The skill-optimization baselines use the same evaluators, target model, optimizer model, and initial skill as SkillFocus. Candidate retention remains method-specific: GEPA keeps its Pareto-based selection, SkillOpt its retained-skill update, and SkillFocus its paired selection and training check. Fairness is enforced through the same task pool and held-out test split, evaluators, target and optimizer models, and initial skills where applicable. Optimization budgets follow each method’s native protocol, while how each method splits the pool and retains candidates remains part of its design. Table 14 summarizes these shared and method-specific conditions.

B.3 Models and Inference Settings

The formal runs use GPT-5.4 as the optimizer model and DeepSeek-V4-Flash as the fixed target model, with random seed 42 and 64 within-run workers. The target model runs with temperature 0 and thinking enabled at low reasoning effort, and optimizer calls use temperature 0. The evolution horizon is at most five rounds with one candidate per round. The same test set is used for final comparison and remains unseen during evolution.

B.4 Evolution Cost Accounting

SkillFocus costs in Table 16 are the sum of Initialization and Iterative Evolution in Table 15. SkillOpt costs are reconstructed by summing prompt and completion tokens over every recorded optimization step in the matched run history, with final test set inference excluded. The relative change is $(C_{\text{evo}}^{\text{SkillFocus}} - C_{\text{evo}}^{\text{SkillOpt}}) / C_{\text{evo}}^{\text{SkillOpt}}$, so negative values indicate a reduction; the mean over benchmarks gives the 24% reduction reported in the main text. Initialization is the one-time pre-evolution bucket: the baseline evaluation and the capability induction that constructs and freezes $(\mathcal{C}, Q)$. Iterative evolution contains per-round diagnosis, edit generation, screening, recomputation of the capability scores, and memo calls. Both buckets constitute C_{evo} ; final test set inference is reported separately as C_{test} and is excluded from C_{evo} . Evolution tokens include model calls incurred before the final skill is fixed, including rejected and abstained can-

didates; a round that attempts more than one capability counts every attempt. Values are rounded to millions of inference tokens.

B.5 Analysis Protocols

The following protocols specify how the ablations and analyses are constructed; their results appear in Section 4 and Appendix A.

B.5.1 Ablation Controls

w/o capability prioritization drops the priority rule and supplies global training set evidence. *w/o matched evidence* keeps the priority rule and supplies global training set evidence. *randomized capability assignment* preserves the task and capability degree sequences while randomizing their correspondence. The three interventions separate where the capabilities come from, which one a round selects, and which evidence the revision reads.

B.5.2 Counterfactual Focus–Evidence Crossover

The focus–evidence crossover is a post-hoc mechanism analysis over the same historical revision states; it does not change capability induction, the evolution horizon, candidate gating, or held-out evaluation. Each unit is a candidate-producing state

$$X_t = (S_t, O_t, \mathbf{q}^t, \mathcal{H}_t; \mathcal{C}, Q).$$

The TM condition reuses the historical top-priority focus and its matched evidence. T→M keeps the top-priority focus but replaces its evidence with the evidence for a code-selected non-top capability c_r . RM uses c_r for both focus and evidence, while R→M uses c_r for focus and the historical top capability’s evidence. The same c_r is used for all three counterfactual branches of a state and is selected without reading their outcomes.

All four conditions share the current skill state, frozen $(\mathcal{C}, Q)$, state-bound task sets, model and prompt configuration, evidence budget, candidate generation, and screening protocol. Counterfactual branches do not write back to the historical run or to one another. The reported acceptance rate, Δ_{sel} , and Δ_{tr} are computed against the same historical current skill, and Table 2 reports macro averages over five states per benchmark. Selection set outcomes remain a screening measure, so the training set candidate gain is the clearest comparison. The crossover measures candidate-level changes from historical states, not held-out test accuracy, and it compares controlled contrasts instead of estimating

Table 14: Shared and method-specific conditions across compared methods. “–” denotes a condition that does not apply to the method.

Condition	No Skill	One-shot Skill	Trace2Skill	GEPA	SkillOpt	SkillFocus
Evaluator and test split	Shared	Shared	Shared	Shared	Shared	Shared
Target model	DeepSeek-V4-Flash	DeepSeek-V4-Flash	DeepSeek-V4-Flash	DeepSeek-V4-Flash	DeepSeek-V4-Flash	DeepSeek-V4-Flash
Initial skill	–	–	Shared	Shared	Shared	Shared
Optimizer	–	–	GPT-5.4	GPT-5.4	GPT-5.4	GPT-5.4
Evolution budget	–	–	One analysis and consolidation pass	160 metric calls	Official setting	At most five rounds, one screened candidate per round
Candidate retention	–	–	Single consolidated skill	Pareto-based selection	Retained-skill update	Positive paired change on selection and training sets
Evolution cost reported	–	–	–	–	Yes	Yes

Table 15: SkillFocus inference-token ledger (millions). SQA, SSB, LM, and IFB abbreviate the four benchmarks; C/A: proposed/accepted candidates; Init., Iter.: initialization and iterative evolution.

Benchmark	C/A	Init.	Iter.	C_{evo}	Test
SQA	5/2	5.10M	15.55M	20.66M	8.51M
SSB	5/2	6.45M	22.61M	29.06M	4.73M
LM	5/2	0.38M	5.98M	6.36M	3.40M
IFB	5/1	0.97M	2.48M	3.45M	1.58M

Table 16: Pre-final-test evolution cost (millions of inference tokens) of SkillFocus and SkillOpt. The final row averages the per-benchmark relative changes.

Benchmark	SkillFocus	SkillOpt	Relative change
SearchQA	20.66	45.21	−54.3%
SpreadsheetBench	29.06	40.55	−28.3%
LiveMath	6.36	8.14	−21.9%
IFBench	3.45	3.21	+7.5%
Mean	–	–	−24.0%

a formal factorial interaction effect. Per-benchmark results are reported in Appendix A.3.

B.5.3 Decomposition-Source Control

To isolate where the capabilities come from, we hold the downstream evolution interface fixed. The SkillFocus condition uses the formal capability space and its endpoint. The task-semantic condition reads the same pre-execution task specifications and describes each task by its overall objective, operation, and domain. The execution-derived condition summarizes how the initial skill behaved on the full non-test task pool.

Let $\mathcal{D}$ denote the task pool after the test set has been removed. The SkillFocus and task-semantic arms induce and freeze $(\mathcal{C}, Q)$ from $\mathcal{D}$ before the

initial skill is executed. The initial skill is then executed on the complete pool,

$$O_0 = \text{Execute}(A, S_0, \mathcal{D}),$$

and $(\mathcal{C}, Q)$ together with O_0 are passed to the shared selection constructor,

$$(D_{\text{sel}}, D_{\text{tr}}) = \text{ConstructSelection}(\mathcal{C}, Q, O_0, M).$$

For the execution-derived condition, this full-pool O_0 supplies the input to the behavior summary, which is frozen as $(\mathcal{C}, Q)$ before the same constructor assigns the two roles. The subsequent evolution uses O_0 restricted to D_{tr} . Every arm forms $(\mathcal{C}, Q)$ before the split, so the training and selection sets follow from the frozen $(\mathcal{C}, Q)$ and O_0 .

Each arm freezes its $(\mathcal{C}, Q)$ once before round 0 and keeps it for the full run. The alternatives are allowed the same number of capabilities, with $K^{\text{sem}} = K^{\text{exec}} = K^{\text{cap}}$, assign several capabilities per task, apply the same support threshold $n_{\text{min}} = 3$, and pass the same natural-language descriptions to revision, assignment and filtering, J_D , and J_G . They share the same task pool and held-out test split, initial skill, target and optimizer models, evaluator, inference settings, evidence budget, prompts, edit operations, history mechanism, five-round horizon, candidate screening, training re-evaluation, and paired acceptance rule. Each condition constructs its own $(D_{\text{sel}}, D_{\text{tr}})$ from its own $(\mathcal{C}, Q)$ and O_0 , so the selection set matches the capability source under test. The primary outcome is final held-out test accuracy, reported in Table 5; token cost and candidate counts are outside this control.

B.5.4 Selection-Set Construction Control

The control keeps the selection set size and replaces the construction of Section 3.4 with uniform random sampling from the same pool on SearchQA, SpreadsheetBench, and LiveMath. The rest of the pipeline is unchanged. Results are reported in Appendix A.6.

C Method Details

This section details each stage of the main-text algorithm, using the notation of Section 3; $\Delta_X(S', S)$ is the paired change of Equation 6. The prompt interfaces behind the model calls are described in Appendix D.

C.1 Capability Induction

Capability induction reads task records from the training and selection sets. Each record contains the task instruction, input structure, and environment contract. The extraction interface never sees the current skill, an execution trace, a success label, a failure explanation, or a candidate edit. The test set remains unavailable until the final retained skill is fixed.

Algorithm C.1 Capability Induction

Require: Task records for the task pool $\mathcal{D}$
Ensure: Frozen capability set $\mathcal{C}$ and matrix Q

- 1: **for all** $\tau_i \in \mathcal{D}$ **do**
- 2: $R_i \leftarrow$ requirements of task i
- 3: **end for**
- 4: **for** $j \leftarrow 1$ to 3 **do**
- 5: $P_j \leftarrow$ proposal from $\{R_i\}$ under ρ_j
- 6: **end for**
- 7: $\mathcal{C}_0 \leftarrow$ consensus of P_1, P_2, P_3
- 8: $Q_0 \leftarrow$ assignment of each R_i to $\mathcal{C}_0$
- 9: **for all** $c_d \in \mathcal{C}_0$ **do**
- 10: **if** $\sum_{\tau_i \in \mathcal{D}} Q_{0,i,d} < n_{\min}$ **then**
- 11: Remove c_d and unassign its requirements
- 12: **end if**
- 13: **end for**
- 14: $(\mathcal{C}, Q) \leftarrow$ freeze $(\mathcal{C}_0, Q_0)$
- 15: **return** $(\mathcal{C}, Q)$

Algorithm C.1 extracts requirements from the task records, forms three independent proposal views, consolidates their registries, and assigns each requirement to one retained capability or UNASSIGNED. Support filtering runs deterministically before $(\mathcal{C}, Q)$ is frozen. The three calls share the same canonical required- behavior input and use fixed Requirement-, Intervention-, and Boundary-centric perspectives, respectively; they do not see one another’s outputs. Here ρ_j denotes the perspective injected for proposal call j .

Algorithm C.2 ConstructSelection

Require: Task pool $\mathcal{D}$, frozen $(\mathcal{C}, Q)$, initial record O_0 , selection size M
Ensure: selection set D_{sel} and training set D_{tr}

- 1: $\pi_d \leftarrow$ support share of c_d in $\mathcal{D}$; $o_i \leftarrow$ diagnosable failure rate of i in O_0 ; $R \leftarrow \{i : O_0 \text{ succeeds on all runs of } i\}$
- 2: $D_{\text{sel}} \leftarrow \emptyset$; $\mathcal{K} \leftarrow \emptyset$
- 3: **while** $|D_{\text{sel}}| < M$ **do**
- 4: $A \leftarrow \{i \in \mathcal{D} \setminus D_{\text{sel}} \text{ that leaves every } c_d \text{ supported outside } D_{\text{sel}}\}$, restricted to R once the remaining slots are needed for the reserve
- 5: $\gamma_i \leftarrow \sum_{d: Q_{i,d}=1, c_d \notin \mathcal{K}} \pi_d$; $\nu_i \leftarrow$ [new features of i]
- 6: $i^* \leftarrow \arg \max_{i \in A} (\gamma_i, \nu_i, o_i) \triangleright$ lexicographic
- 7: $D_{\text{sel}} += \{i^*\}$; $\mathcal{K} += \{c_d : Q_{i^*,d} = 1\}$
- 8: **end while**
- 9: **return** frozen sets $(D_{\text{sel}}, \mathcal{D} \setminus D_{\text{sel}})$

C.2 Selection-Set Construction

Algorithm C.2 builds the selection set of Section 3.4. The coverage weight π_d is the share of pool tasks whose row in Q marks c_d , so the same support share that weights unresolved task mass in Equation 4 also weights coverage here. Diversity counts the static task features an item adds to the partial selection set, using the task family, input structure, and environment contract labels recorded with each item. The improvement-opportunity term is the fraction of an item’s initial executions that fail for diagnosable task-logic reasons. That classification is deterministic and uses no model call: failed executions are grouped first by the evaluator’s error signal and then by the repair signature of the trace, and failures caused by the execution environment are counted apart from them, so environment faults win no slots. Items solved on every initial execution form the stable-success reserve, which the construction fills to one fifth of the selection set once the remaining slots would otherwise leave it unreachable; this fraction is fixed in advance and is not tuned against any outcome. The feasibility constraint keeps, for every capability, at least one supporting task in the training set. It reads Q alone, so the task it keeps need not be one the initial skill fails, and a capability whose failing tasks all move to the selection set leaves the eligible capability set of Equation 4 for the rest of the run. Ties at every level are broken by task identifier, which makes the construction deterministic given the pool, $(\mathcal{C}, Q)$, and the initial execution record.

C.3 Capability Scores and Capability Selection

Algorithm C.3 recomputes the capability scores from the training set and orders the eligible capabil-

Algorithm C.3 Capability Scores and Capability Selection

Require: A , skill S_t , record O_t , D_{tr} , D_{sel} , $\mathcal{C}$, Q
Ensure: Record O_t , capability scores $\mathbf{q}^t$, ordered eligible capabilities $\mathcal{U}_t$

- 1: **if** O_t is unavailable **then**
- 2: $O_t \leftarrow$ execute A with S_t on D_{tr}
- 3: **end if**
- 4: **for all** $c_d \in \mathcal{C}$ **do**
- 5: $\pi_d \leftarrow \frac{1}{|D_{tr} \cup D_{sel}|} \sum_{i \in D_{tr} \cup D_{sel}} Q_{i,d}$
- 6: $q_d^t \leftarrow$ mean of $Y_i(S_t)$ over $i \in D_{tr}$ with $Q_{i,d} = 1$
- 7: $m_d^t \leftarrow \pi_d (1 - q_d^t)$
- 8: **end for**
- 9: $\mathcal{U}_t \leftarrow \{c_d : \text{diagnosable failure in } D_{tr}, \text{ support in } D_{sel}\}$
- 10: **return** ($O_t, \mathbf{q}^t, \mathcal{U}_t$ by decreasing m_d^t)

ities by unresolved task mass, with ties broken by registry order; a round tries them in this order until one yields a candidate. A capability whose last candidate was rejected for a confirmed non-positive paired effect sits out the next round, and returns at once when no other capability is eligible; in the reported runs every round produced a candidate for its first eligible capability. An accepted candidate's re-evaluation becomes the next O_t .

C.4 Capability-Guided Revision

For the selected capability, retrieval keeps the training set tasks that Q marks for it. The bundle contains failures on those tasks, successful executions to compare against, related current-skill content, and earlier edits made for the same capability. Failure examples are stratified by observed reason, duplicate traces are removed, and retrieval order rotates across rounds when more examples are available than the evidence allowance. A failed task is paired first with a successful execution of the same task when available, otherwise with a successful task requiring the selected capability. When the record holds no successful execution for that capability, the diagnosis is told so instead of receiving a substitute contrast.

Algorithm C.4 makes the evidence boundary explicit: the selected capability picks the failures and success contrasts, pulls in the related skill content and its own revision history, and then frames diagnosis and edit integration. The edit is applied to the complete skill. Before execution, a candidate is discarded if it cites evidence outside its bundle, targets an instruction the skill does not contain, duplicates an existing instruction, or copies task-specific entities such as task identifiers, file or sheet names, and target coordinates. A removal is admitted only when observations across several rounds and tasks associate the targeted instruction with

Algorithm C.4 Capability-Guided Revision

Require: Selected capability c_t^* , skill S_t , record O_t , history $\mathcal{H}, Q$
Ensure: Candidate S'_t and record r_t , or ABSTAIN

- 1: $I_t \leftarrow \{i \in D_{tr} : Q_{i,d_t^*} = 1\}$
- 2: $\mathcal{E}_t \leftarrow$ failures and successes on I_t in O_t
- 3: Add skill content related to c_t^* to $\mathcal{E}_t$
- 4: Add history entries of c_t^* in $\mathcal{H}$ to $\mathcal{E}_t$
- 5: **for** $j \leftarrow 1$ to J_D **do**
- 6: $z_t^{(j)} \leftarrow$ diagnosis from $\mathcal{E}_t$ under π_j^D
- 7: **end for**
- 8: $z_t \leftarrow$ integration of $\{z_t^{(j)}\}$ against $\mathcal{E}_t$
- 9: **if** z_t is not actionable **then**
- 10: **return** (ABSTAIN, $r_t = (z_t)$)
- 11: **end if**
- 12: **for** $j \leftarrow 1$ to J_G **do**
- 13: $g_t^{(j)} \leftarrow$ edit concept for z_t under π_j^G
- 14: **end for**
- 15: $e_t \leftarrow$ integration of $\{g_t^{(j)}\}$ into one edit
- 16: **if** no valid edit exists **then**
- 17: **return** (ABSTAIN, $r_t = (z_t)$)
- 18: **end if**
- 19: $S'_t \leftarrow S_t$ with e_t applied; $r_t \leftarrow (z_t, e_t)$
- 20: **return** (S'_t, r_t)

the failing behavior. These checks are deterministic and add no model calls. In the formal runs, $J_D = J_G = 3$. The value follows the three frozen operational perspectives of each stage, with one independent Explorer per perspective, and was set without reference to benchmark outcomes; sensitivity to the number of Explorers is left to future work. Each Explorer call is independent, uses its corresponding configured perspective π_j^D or π_j^G , and is followed by one Integrator call for that stage. The independent-view design follows the logic of self-consistency (Wang et al., 2023); its multi-agent consolidation step is also related to debate-based aggregation (Du et al., 2024).

C.5 Candidate Screening

Candidate screening compares the candidate and the current skill on identical task instances.

Algorithm C.5 Candidate Screening and Skill Update

Require: A , skill S_t , candidate S'_t , record O_t , D_{sel} , D_{tr}
Ensure: S_{t+1} , O_{t+1} , decision a_t , and record u_t

- 1: $\Delta_{sel}^t \leftarrow \Delta_{D_{sel}}(S'_t, S_t)$
- 2: **if** $\Delta_{sel}^t \leq 0$ **then**
- 3: **return** (S_t, O_t , REJECT, $u_t = (\Delta_{sel}^t)$)
- 4: **end if**
- 5: $O'_t \leftarrow$ execute A with S'_t on D_{tr}
- 6: $\Delta_{tr}^t \leftarrow \Delta_{D_{tr}}(S'_t, S_t)$
- 7: $u_t \leftarrow (\Delta_{sel}^t, \Delta_{tr}^t)$
- 8: **if** $\Delta_{tr}^t > 0$ **then**
- 9: **return** (S'_t, O'_t , ACCEPT, u_t)
- 10: **end if**
- 11: **return** (S_t, O_t , REJECT, u_t)

Algorithm C.5 screens a complete candidate first on the selection set and then, after a passing first

Table 17: Overview of the prompt interfaces and their call multiplicity in the formal runs.

Interface	Role	Calls
<i>capability induction</i>		
ExtractRequirements	task $\rightarrow$ requirements	per task
ProposeCapabilities	cross-task proposals	3
Consensus	frozen registry	1
Assign	requirement $\rightarrow$ capability	batched
<i>capability revision</i>		
ExploreDiagnoses	diagnosis hypotheses	3/round
IntegrateDiagnosis	selected diagnosis	1/round
ExploreEdits	edit concepts	3/round
IntegrateEdit	final candidate edit	1/round

Table 18: Frozen Explorer perspectives in the formal runs.

Stage / perspective	Frozen operational condition
Diagnosis: missing guidance	No current-skill rule addresses the operation; the explorer leaves <code>cited_rules</code> empty.
Diagnosis: ineffective guidance	An existing rule addresses the operation but fails; the explorer quotes at least one such rule.
Diagnosis: behavior organization	Interacting rules create an ordering, conditioning, or precedence problem; the explorer quotes at least two rules.
Generation: minimal addition	Propose one non-duplicative standalone rule with <code>op=add</code> .
Generation: revise existing guidance	Replace the core content of one existing rule, quoting the rule being revised.
Generation: structural change	Change a rule’s trigger, ordering, precedence, interaction, or removal target, quoting the affected rule(s).

stage, on the training set. Only an accepted candidate replaces the current skill and supplies the next training set record. After each round, the loop appends the selected capability, the revision record, and the screening outcome (c_t^*, r_t, u_t) to $\mathcal{H}$. Abstentions and rejected candidates are retained as revision history, while leaving the current skill unchanged.

D Representative Prompt Interfaces

SkillFocus issues its model calls through eight structured prompt interfaces (Table 17), with runtime records, skill text, evidence, history, and perspectives substituted by the runner. We show one interface per stage; the remaining five follow the same contract.

The runner selects these rows in order, so π_j^D and π_j^G in Algorithm C.4 identify the row for Explorer call j ; the configured *mechanism-grounded addition* perspective is unused at $J_G = 3$.

D.1 Capability Proposal

Interface: `ProposeCapabilities`. Group required behaviors that recur across distinct tasks and can share skill guidance. Keep each capability cross-task, observable, and skill-addressable; do not group by benchmark, tool, operation name, or output artifact. The shared template is called with one fixed perspective tag ρ_j per independent proposal. Specifically, (ρ_1, ρ_2, ρ_3) are REQUIREMENT-CENTRIC, INTERVENTION-CENTRIC, and BOUNDARY-CENTRIC; the tag changes the grouping lens but not the input records or output schema. The proposals use only the canonical required-behavior records and do not use execution outcomes, current skill text, or evaluator results. Merge behaviors only when the same reusable guidance would normally improve them; keep them separate when distinct guidance is needed. Do not optimize for a target number of capabilities, and cite representative behavior records rather than performing assignment in this call.

Output.

```
<capability_proposal>
  <capabilities>
    <capability>
      <name>behavioral name</name>
      <definition>
        reusable requirement
      </definition>
      <evidence_refs>
        <ref>RB_ID</ref>
      </evidence_refs>
    </capability>
  </capabilities>
</capability_proposal>
```

D.2 Diagnosis Explorer

Interface: `ExploreDiagnoses`. Given the selected capability, evidence bundle, and current skill, develop one evidence-supported hypothesis connecting an observed execution pattern to an editable skill location. One Explorer returns one hypothesis; it does not write an edit. The Explorer cites the evidence records and any current-skill rules required by its configured perspective, and abstains when the bundle cannot support one defensible hypothesis.

Output.

```
<diagnosis_explorer_result>
  <status>
    supported|insufficient_evidence
  </status>
  <hypothesis>
    one evidence-grounded diagnosis
  </hypothesis>
  <cited_rules>
    <rule>verbatim current-skill rule</rule>
  </cited_rules>
  <evidence_refs>
    <ref>EVIDENCE_ID</ref>
  </evidence_refs>
</diagnosis_explorer_result>
```

D.3 Edit Integrator

Interface: `IntegrateEdit`. Select or synthesize the smallest coherent edit from the proposed concepts and the diagnosis. Return `no_valid_edit` if no operation has a precise target and a reusable content change. The returned candidate records its operation, target contract, preservation scope, trigger/action/check, expected change, falsifier, and history

assessment in addition to the target-facing content. The integrator also checks the formal rejection memo, preserves effective unrelated skill content, avoids benchmark-specific hard-coding, and requires a concrete worked example in the target-facing edit.

Output.

```
<candidate_edit_result>
  <status>
    candidate|no_valid_edit
  </status>
  <op>add|replace|remove</op>
  <target>
    exact target or insertion location
  </target>
  <trigger>
    when the rule applies
  </trigger>
  <action>
    what the agent should do
  </action>
  <check>
    what the agent should verify
  </check>
  <preserve>
    effective behavior to preserve
  </preserve>
  <expected_change>
    failure behavior expected to change
  </expected_change>
  <content><![CDATA[
exact target-facing Skill edit including
the worked example
]]></content>
</candidate_edit_result>
```

Audit fields for the target specification, preservation scope, history assessment, falsifier, and rationale are omitted.

E Use of Large Language Models

Large language models are components of the studied system: GPT-5.4 is the optimizer model for capability induction, diagnosis, and edit generation, and DeepSeek-V4-Flash is the fixed target model that executes benchmark tasks; the baselines use the same models in these roles.